MAJOR COURSE OUTPUT 1 (MCO 1) • TECHNICAL PROJECT REPORT

BantayPetsa

Bantay sa petsa. Alaga sa paninda.

Course: Mobile Programming 1 (Instructor: Aaron Jude C. Pael)

Platform: Android (Kotlin 2.0 • Jetpack Compose • Material 3)

Architecture: MVVM • Unidirectional Data Flow (UDF) • Lifecycle-Aware

PROJECT OVERVIEW

Problem Addressed

Small Philippine retail micro-merchants (sari-sari stores, mini-groceries) face acute operational friction in inventory and expiration control:

- **Manual, Error-Prone Shelf Monitoring:** Store owners rely on unorganized memory or handwritten notebooks, causing batch expiration dates to be overlooked until past maturity.
- **Direct Financial Losses (*Tapon* / Wastage):** Overdue consumables (milk, bread, canned goods) cannot be sold, resulting in unrecoverable inventory write-offs that directly harm fragile micro-retail profit margins.
- **Consumer Safety & Trust Risks:** Inadvertently leaving spoiled or expired goods on store display racks compromises customer safety, invites disputes, and permanently damages community trust.
- **Absence of First Expired, First Out (FEFO) Rotation:** Multiple product deliveries with varying expiry dates end up mixed together on shelves, lacking deterministic priority on which units to sell first.
- **Zero Traceable Stock Audit Trail:** Store owners lack historical logs to evaluate whether stock losses were caused by customer theft, supplier spoilage, or unsellable stock.

Target Users

Designed specifically for resource-constrained, fast-paced retail environments:

- **Sari-Sari Store Owners & Neighborhood Vendors:** Sole proprietors requiring simple, one-handed, offline-first batch logging with glanceable daily alerts.
- **Mini-Grocery & Convenience Store Operators:** Micro-retailers handling high volumes of fast-moving packaged food, dairy, baby formula, and canned items across distinct intake dates.
- **Store Cashiers, Tills & Helpers:** Store staff needing unambiguous visual status badges (Safe, Approaching, Urgent, Expired) to immediately identify which items to prioritize or bundle for markdown.

CORE FEATURES (IMPLEMENTED IN MCO 1)

1. **Product & Batch Registration (with Catalog Archiving)**
   Record products with custom measurement units (e.g., cans, sachets, packs). Attach multiple batches with separate expiration dates to the same product, and easily **Archive** discontinued products to hide them from the active list without losing past history.

2. **Automated 4-Tier Expiry Status Engine**
   Automatically calculates remaining days relative to the phone's calendar, categorizing each batch into clear operational tiers without manual math:

Safe	Approaching	Urgent	Expired
> 30 Days Left	8 to 30 Days Left	0 to 7 Days Left	< 0 Days Left
Fresh stock; standard shelf display.	Move stock forward to the front.	Immediate bundle, markdown, or sale.	Pull from shelves immediately.

3. **Deterministic First Expired, First Out (FEFO) Prioritization**
   Organizes stock so that **whatever expires first appears at the top** (`Expiry Date ↑ → Intake Date ↑ → Batch ID ↑`), ensuring older stock is sold before newer deliveries.

4. **Stock Movement Actions (Deduct, Add, and Set / Audit)**

   Provides 3 core inventory actions: **Deduct** (sales, returns, or disposal), **Add** (restock delivery), and **Set / Audit** (reconciling physical shelf count with the app). Strictly enforces non-negative invariants ($Q_{after} = Q_{before} + \Delta Q \geq 0$).

5. **Dynamic History & Time-Filtered Reports**

   Maintains a complete audit trail of every stock action. A live report card recalculates total items restocked, sold/deducted, and net movement across *All Time*, *Last 7 Days (Weekly)*, and *Last 30 Days (Monthly)*.

6. **Glanceable Dashboard & Quick Alerts**

   At-a-glance overview screen showing 2x2 status summary metric cards, percentage distribution indicators, active catalog counters, and a priority list of items needing immediate sale.

7. **100% Offline-First Architectural Autonomy**

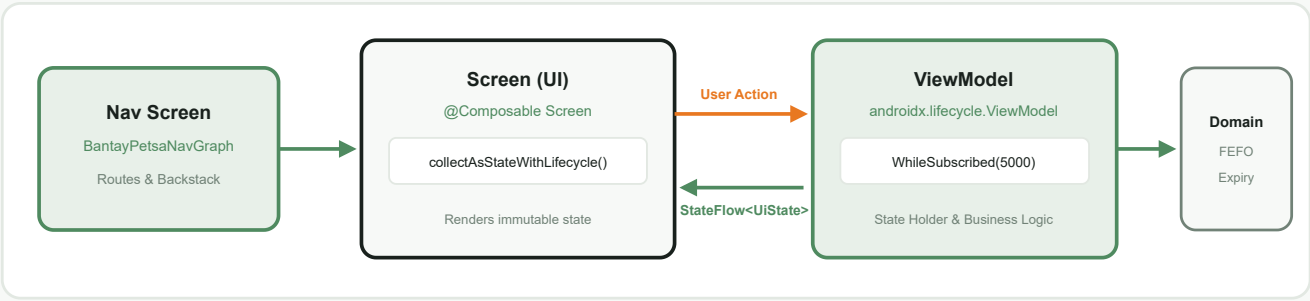
   Operates completely without internet connectivity, external APIs, or cloud dependencies, ensuring store owners can manage inventory reliably anywhere in the Philippines.

TECHNICAL SCOPE IMPLEMENTATION (MCO 1)

Core Technologies: Kotlin 2.0, Jetpack Compose & Material 3

The application is built 100% in modern **Kotlin 2.0.21** utilizing declarative **Jetpack Compose** (BOM 2024.10.00). Declarative UI eliminates legacy XML layouts by writing composable functions that automatically re-render when state changes. Styled with **Material 3** design tokens, featuring the customized *Tarsi Green* palette (`PrimaryGreen #4F8A60`, `PaleGreen #E8F0E9`, `SurfaceWhite #FFFFFF`), dynamic status badges, and Material typography hierarchy.

Unidirectional Data Flow (UDF) & Lifecycle Architecture



1. UI State (Unidirectional Data Flow)

Concept: An explicit, immutable data class representing a single snapshot of the screen. The UI only reads this state and cannot modify it directly.
Benefit: Eliminates race conditions and UI glitches because data flows in strictly one direction.

```
data class ProductsUiState(
    val searchQuery: String = "",
    val selectedFilter: String =
"Active",
    val products: List<ProductItem> =
emptyList(),
    val batches: List<BatchItem> =
emptyList(),
    val isLoading: Boolean = false
)
```

2. Nav Screen (Navigation Compose)

Concept: Central navigation host (`BantayPetsaNavGraph`) coordinating 5 core bottom tabs and sub-screens with typed argument passing and backstack control.
Benefit: Prevents duplicate screen stacking (`launchSingleTop`), preserves state during tab switching, and guarantees clean back navigation.

```
// BantayPetsaNavGraph.kt
composable(Screen.Home.route) {
    DashboardScreen(
        onNavigateToProducts = {
            navController.navigate(Screen.Products.route)
{

popUpTo(navController.graph.findStartDestination().id) {
saveState = true }
                launchSingleTop = true; restoreState =
true
            }
        },
        onNavigateToAlerts = { filter ->

navController.navigate(Screen.Alerts.createRoute(filter))
        }
    )
}
```

3. ViewModel Logic

Concept: `androidx.lifecycle.ViewModel` acts as the screen's brain. It holds business calculations, transforms raw flows into UI state, and survives configuration changes.
Benefit: If the merchant rotates their phone, the screen does not reset or reload; the ViewModel preserves state seamlessly in memory.

```
// ProductsViewModel.kt
class ProductsViewModel : ViewModel() {
    private val _searchQuery =
MutableStateFlow("")
    private val _selectedFilter =
MutableStateFlow("Active")

    val uiState:
StateFlow<ProductsUiState> = combine(
        SampleInventoryState.products,
_searchQuery, _selectedFilter
    ) { products, query, filter ->
        val filtered = products.filter {
            (filter == "All" ||
(!it.isArchived && filter == "Active")
|| (it.isArchived && filter ==
"Archived")) &&
            (query.isBlank() ||
it.name.contains(query, ignoreCase =
true))
        }
        ProductsUiState(searchQuery =
query, selectedFilter = filter, products
= filtered)
    }.stateIn(viewModelScope,
SharingStarted.WhileSubscribed(5000),
ProductsUiState())
}
```

4. Lifecycle-Aware Collection

Concept: Connecting composables to the ViewModel using `collectAsStateWithLifecycle()` from `androidx.lifecycle.runtime.compose`.
Benefit: Automatically pauses flow emissions 5 seconds after a screen enters the background (`WhileSubscribed(5000)`), saving battery and preventing memory leaks.

```
// ProductsScreen.kt
@Composable
fun ProductsScreen(
    onProductClick: (String) -> Unit,
    viewModel: ProductsViewModel = viewModel()
) {
    // Lifecycle-aware: pauses in background, resumes
when visible
    val uiState by
viewModel.uiState.collectAsStateWithLifecycle()

    // UI purely consumes immutable uiState
    LazyColumn {
        items(uiState.products, key = { it.productId }) {
product ->
            ProductListItemCard(product = product,
onClick = { onProductClick(product.productId) })
        }
    }
}
```

PLANNED FOR MCO 2 (FINAL MILESTONE)

While MCO 1 established the complete UI, navigation, ViewModel logic, and domain algorithms using in-memory state, **Major Course Output 2 (MCO 2)** will introduce full offline persistence and owner customization:

1. Room SQLite Database (Persistent Offline Storage)

Architecture: Transitioning from in-memory singletons to persistent local SQLite storage using Android Jetpack's **Room Persistence Library** (`@Entity`, `@Dao`, `@Database`, `@Transaction`).

Operational Purpose: Guarantees that registered products, batches, and stock action audit history survive device reboots, app restarts, process death, and extended airplane mode. Atomic `@Transaction` methods will ensure that whenever a batch is deducted or added, the corresponding historical action entry is written simultaneously, preventing data desynchronization.

2. Custom Warning Thresholds (Preferences DataStore)

Architecture: Implementing **Jetpack Preferences DataStore** to store user-defined warning configurations asynchronously without blocking the UI thread.

Operational Purpose: Different retail categories require different operational thresholds. For example, fresh panaderia baked goods spoil within 3 days, whereas canned corned beef has a 2-year lifespan. Through the Settings screen, the store owner can customize their own warning periods (e.g., setting *Urgent* from 7 to 3 days, and *Approaching* from 30 to 14 days), dynamically reconfiguring the `ExpiryCalculator` engine.

OTHERS: DOMAIN LOGIC IMPLEMENTATION & SAMPLE DATA

The underlying domain engines implemented during Phase 3 are pure Kotlin components decoupled from Android frameworks for maximum reliability, speed, and testability:

A. Deterministic FEFO (First Expired, First Out) Sorting Engine

`app/src/main/java/com/bantaypetsa/app/domain/engine/FefoEngine.kt`

Simple Explanation: Sorts all inventory batches strictly by expiration date (earliest first). If two batches expire on the exact same date, the older delivery intake date is used as a tie-breaker, guaranteeing deterministic stock rotation.

```
fun sortBatches(batches: List<BatchItem>): List<BatchItem> {
    return batches.sortedWith(
        compareBy<BatchItem> { it.expirationDate }  // 1. Earliest expiry first
            .thenBy { it.dateReceived }             // 2. Oldest intake tie-breaker
            .thenBy { it.createdAt }                // 3. Creation timestamp tie-breaker
            .thenBy { it.batchId }                  // 4. Deterministic fallback
    )
}
```

B. Automated 4-Tier Expiry Status Engine

`app/src/main/java/com/bantaypetsa/app/domain/engine/ExpiryCalculator.kt`

Simple Explanation: Mathematical function that compares the batch's expiration date against the current date and instantly assigns one of the 4 operational tiers: *Expired* (<0d), *Urgent* (0–7d), *Approaching* (8–30d), or *Safe* (>30d).

```
fun calculateStatus(expirationDate: LocalDate, referenceDate: LocalDate = LocalDate.now()): ExpiryStatus
{
    val days = ChronoUnit.DAYS.between(referenceDate, expirationDate)
    return when {
        days < 0 -> ExpiryStatus.EXPIRED         // Past expiration date
        days <= 7 -> ExpiryStatus.URGENT         // 0 to 7 days left (bundle/markdown)
        days <= 30 -> ExpiryStatus.APPROACHING   // 8 to 30 days left (front shelf)
        else -> ExpiryStatus.SAFE                // More than 30 days left (fresh)
    }
}
```

C. Stock Invariant Arithmetic Engine

`app/src/main/java/com/bantaypetsa/app/domain/engine/StockInventoryEngine.kt`

Simple Explanation: Enforces retail inventory mathematical integrity. Handles Add (restock), Deduct (sale/disposal), and Set/Audit (physical inventory count), strictly blocking deductions that exceed current on-hand stock ($Q_{after} \geq 0$).

```
when (actionType) {
    StockActionType.ADD -> Success(quantityAfter = currentQuantity + inputQuantity)
    StockActionType.REMOVE -> {
        if (currentQuantity < inputQuantity) Error("Insufficient stock on hand")
        else Success(quantityAfter = currentQuantity - inputQuantity)
    }
    StockActionType.ADJUST -> Success(quantityAfter = inputQuantity) // Set / Audit physical shelf count
}
```

D. In-Memory Sample / Dummy Data Storage

`app/src/main/java/com/bantaypetsa/app/core/util/SampleInventoryState.kt`

Simple Explanation: For MCO 1, sample data is stored in a centralized in-memory Kotlin singleton using `StateFlow`. It is pre-seeded with realistic Philippine sari-sari store products (*Bear Brand Fortified, Lucky Me! Pancit Canton, San Miguel Pale Pilsen, Mega Sardines*) and batches spanning varied expiry dates (expired, urgent, approaching, safe) to provide a fully functional interactive demonstration until Room DB persistence is connected in MCO 2.

```
object SampleInventoryState {
    private val _products = MutableStateFlow<List<ProductItem>>(initialProducts)
    val products: StateFlow<List<ProductItem>> = _products.asStateFlow()

    private val _batches = MutableStateFlow<List<BatchItem>>(initialBatches)
    val batches: StateFlow<List<BatchItem>> = _batches.asStateFlow()
}
```